

Implementasi Supervised Learning pada dataset *Synthetic Social Media Messages for Early Cyberattack Detection on Blockchain*

Disusun untuk memenuhi tugas 3 mata kuliah Pembelajaran Mesin

Oleh:

Willy Jonathan Arsyad	(2208107010037)
Agil Mughni	(2208107010025)
Alfi Zamriza	(2208107010080)
T.M Fadlul Ihsan	(2208107010088)
M. Arkan Haris	(2208107010022)



**JURUSAN INFORMATIKA
FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
UNIVERSITAS SYIAH KUALA
DARUSSALAM, BANDA ACEH
2025**

Daftar Isi

Daftar Isi	1
1. Data Description	2
2. Data Loading	2
3. Data Understanding	2
4. Data Preprocessing	6
5. Model Implementation	8
6. Training Model	9
7. Model Evaluation	12

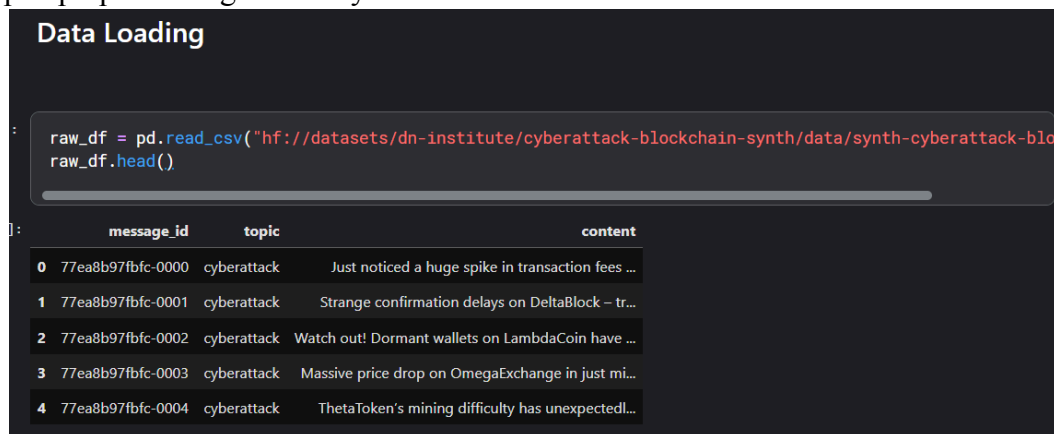
1.Data Description

Mengenai dataset *Synthetic Social Media Messages for Early Cyberattack Detection on Blockchain*, yang diambil dari website [hugging face](#), dan merupakan dataset text sintetik mengenai pesan sosial media pada topik *blockchain*. dari publikasi [ELTEX: A Framework for Domain-Driven Synthetic Data Generation](#) dataset tersebut terdiri dari 3 kolom dan 11.465 baris data dalam format file 'csv'. Dimana dari ke-3 kolom dari dataset tersebut berupa:

1. message_id → ID unik masing-masing pesan yang dikirim
2. topic → Topik pesan yang dikirim (Terdapat 2 topik, 'cyberattack' dan 'general')
3. content → Isi dari pesan yang dikirimkan

2.Data Loading

Dataset diolah dengan menggunakan python pada environment kaggle code, sehingga pembacaan data dilakukan dengan membaca dataset dengan perantara huggingface-cli dan membaca file 'csv' dari kaggle yang telah diupload. Dataset dibaca menggunakan library 'pandas' dan disimpan dalam variabel untuk tahapan preprocessing berikutnya.



```
raw_df = pd.read_csv("hf://datasets/dn-institute/cyberattack-blockchain-synth/data/synth-cyberattack-blo")
raw_df.head()
```

	message_id	topic	content
0	77ea8b97fbfc-0000	cyberattack	Just noticed a huge spike in transaction fees ...
1	77ea8b97fbfc-0001	cyberattack	Strange confirmation delays on DeltaBlock - tr...
2	77ea8b97fbfc-0002	cyberattack	Watch out! Dormant wallets on LambdaCoin have ...
3	77ea8b97fbfc-0003	cyberattack	Massive price drop on OmegaExchange in just mi...
4	77ea8b97fbfc-0004	cyberattack	ThetaToken's mining difficulty has unexpectedl...

3.Data Understanding

Pada tahapan pemrosesan ini ada cukup banyak hal untuk dilakukan, namun kami mulai dengan proses pengecekan data yang telah dibaca sebelumnya. Dataset dicek kembali jumlah baris, dan kolom yang terbaca, yaitu 11.465 baris, dengan 3 kolom. Kemudian, kami menampilkan tipe data dari masing-masing kolom dataset yang telah dibaca, dan memastikan bahwa tipe data yang ditetapkan secara otomatis memang sudah tepat.

```

: raw_df.shape
: (11465, 4)

raw_df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 11465 entries, 0 to 11464
Data columns (total 3 columns):
#   Column      Non-Null Count  Dtype
---  -
0   message_id  11465 non-null  object
1   topic       11465 non-null  object
2   content     11465 non-null  object
dtypes: object(3)
memory usage: 268.8+ KB

```

Kemudian, kami melakukan analisis terhadap dataset tersebut. Diantaranya yaitu melihat unique value dari setiap kolom dataset, dan juga label dengan jumlah kemunculan terbanyak dari masing-masing kolom pada dataset.

```
raw_df.describe()
```

	message_id	topic	content
count	11465	11465	11465
unique	11465	2	11465
top	cffbaac175fe-0095	cyberattack	Blockchain collaborations are driving some coo...
freq	1	6941	1

Selanjutnya, kami juga memeriksa jumlah null value, ataupun data dengan entry yang sama (duplikat). Namun setelah pemeriksaan, tidak ditemukan adanya baris dengan multiple entry ataupun memiliki null value, sehingga dataset ini tidak kami lakukan pengurangan atau pembuangan baris dari dataset, ataupun penyesuaian lainnya seperti pengisian nilai dari kolom berdasarkan nilai rata-rata, atau median.

```
raw_df.duplicated().sum()
```

0

```
raw_df.isna().sum()
```

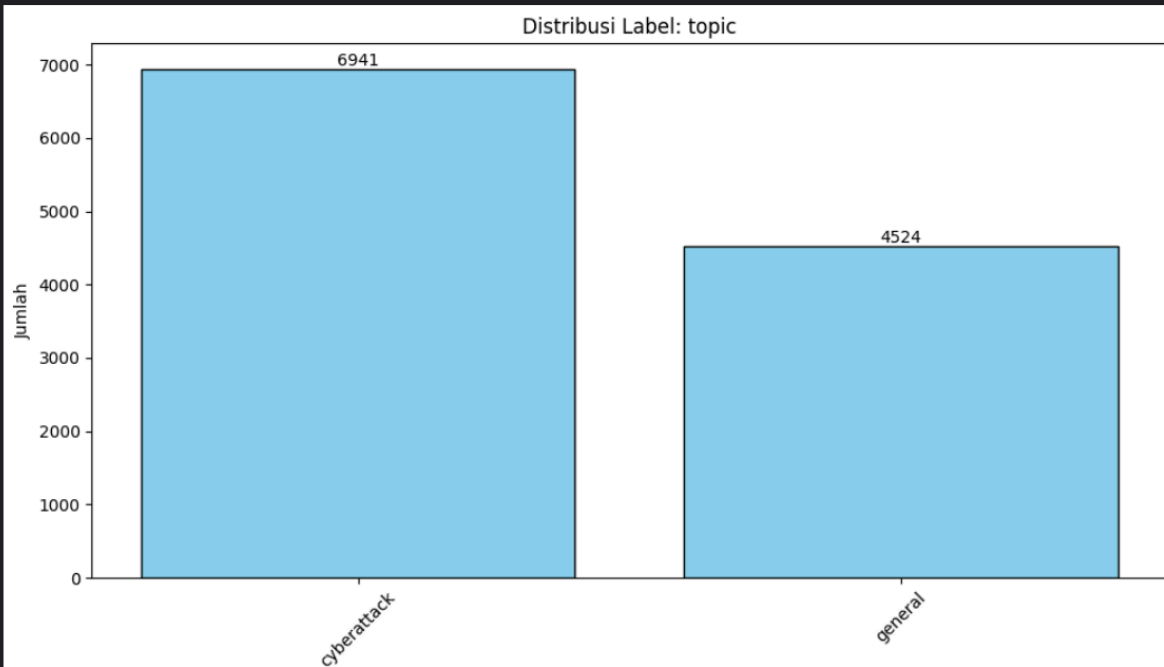
```
message_id    0  
topic          0  
content        0  
word_count     0  
dtype: int64
```

Berdasarkan hasil, terlihat bahwa dataset ini tidak memiliki data yang terduplikat ataupun kolom dengan data null. Kita dapat melanjutkan ke proses analisis selanjutnya.

Berdasarkan bar graph yang ditampilkan, terdapat 2 jenis label pada kolom topik. label 'cyberattack' dengan jumlah kemunculan terbanyak pada 6941 baris, dan label 'general' dengan 4524 baris. label 'cyberattack' merupakan data yang memiliki topik percakapan Sinyal peringatan dini dan indikator *cyber attack*, sedangkan label general merupakan data dengan topik diskusi blockchain pada umumnya yang tidak berkaitan dengan keamanan. Berdasarkan jumlah tersebut, dataset ini dapat dikatakan relatif seimbang, dimana label ke-dua dataset tidak memiliki perbedaan yang terlalu signifikan.

```
def plot_label_distribution(df, label_col, top_n=15):  
    if label_col not in df.columns:  
        print(f"Kolom {label_col} tidak ditemukan dalam DataFrame.")  
        return  
  
    # Hitung frekuensi label  
    label_counts = df[label_col].value_counts().nlargest(top_n)  
    labels = label_counts.index  
    counts = label_counts.values  
  
    # Plot  
    plt.figure(figsize=(10, 6))  
    bars = plt.bar(labels, counts, color='skyblue', edgecolor='black')  
  
    # Tambahkan nilai di atas bar  
    for bar, count in zip(bars, counts):  
        plt.text(bar.get_x() + bar.get_width() / 2, bar.get_height(), str(count),  
                 ha='center', va='bottom', fontsize=10)  
  
    plt.title(f'Distribusi Label: {label_col}')  
    plt.xlabel(label_col)  
    plt.ylabel('Jumlah')  
    plt.xticks(rotation=45)  
    plt.tight_layout()  
    plt.show()
```

```
# Menampilkan distribusi label 'topic'
plot_label_distribution(raw_df, 'topic')
```



Berdasarkan grafik yang kami tampilkan sebelumnya, kami menyimpulkan bahwa dataset ini sudah cukup dilakukan preprocessing. Kami juga memutuskan untuk menambahkan kolom baru yang berisi jumlah kata, untuk membantu algoritma yang akan kami gunakan nantinya dalam melatih dan memprediksi dataset tersebut. Pada kolom yang baru kami tambahkan ini, memiliki secara rata-rata sekitar 11 kata. dengan Jumlah kata minimal yaitu 3 dan maksimal yaitu 22 kata dalam 1 pesan yang dikirim. Adapun standar deviasi dari kolom ini yaitu berada di sekitar 2.5.

```
# Menambahkan kolom baru berisi jumlah kata
raw_df['word_count'] = raw_df['content'].apply(lambda x: len(str(x).split()))

# Cek deskriptif dari kolom word_count
raw_df['word_count'].describe()
```

```
count    11465.000000
mean      11.071871
std        2.578915
min         3.000000
25%         9.000000
50%        11.000000
75%        13.000000
max        22.000000
Name: word_count, dtype: float64
```

4.Data Preprocessing

Berikutnya kami lakukan tahapan Data Preprocessing, dimana dataset text ini akan kami ubah menjadi numerik dengan memanfaatkan one-hot encoding, embedding data text dari pesan menggunakan model all-mpnet-base-v2. Ada juga beberapa kolom yang kami ubah namanya seperti kolom 'topic' menjadi 'class' dan menghapuskan kolom 'message_id' yang tidak diperlukan dalam melatih model dalam melakukan prediksi.

```
df = raw_df.copy()

df.drop(columns='message_id', inplace=True)

df.rename(columns={
    'topic': 'class'
}, inplace=True)
```

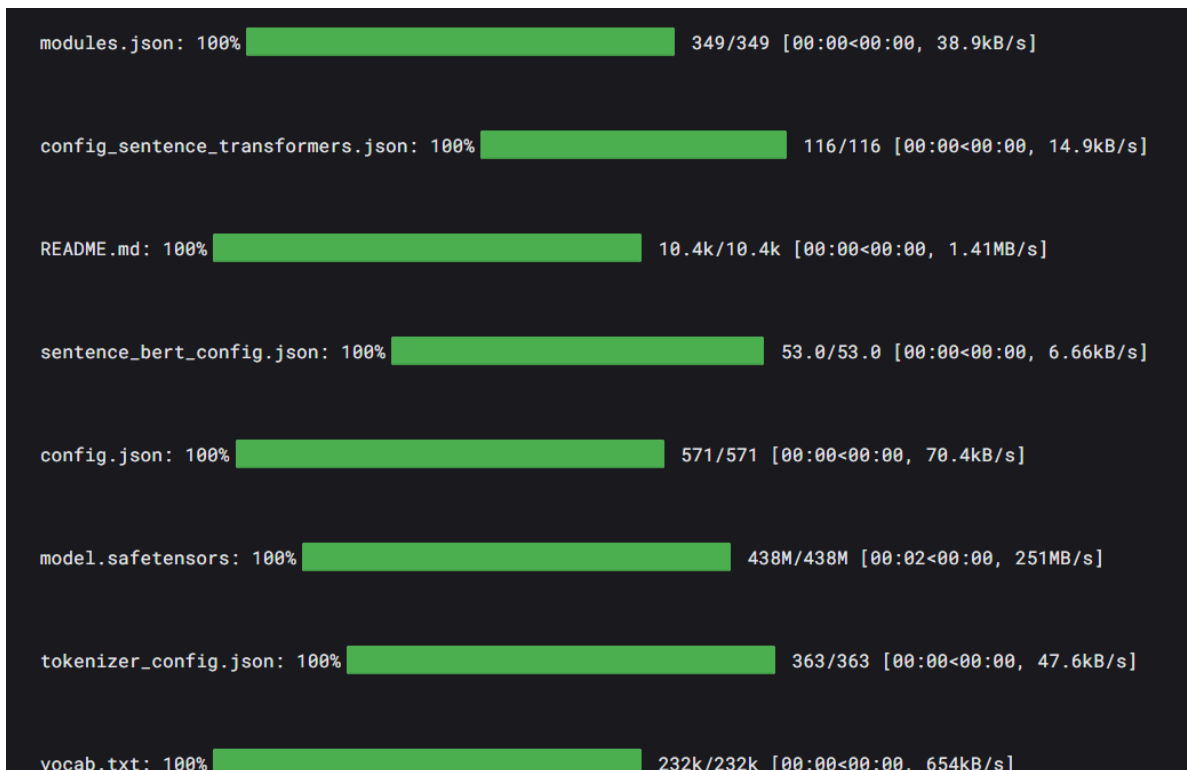
Kemudian, dari dataframe baru yang telah kami buat, dilakukan proses embedding menggunakan model *mpnet*.

Embedding Column *content*

```
def process_series_embedding(series, model, batch_size=32):
    texts = series.tolist()
    embeddings = model.encode(texts, batch_size=batch_size, show_progress_bar=True, convert_to_numpy=True)
    return embeddings
```

Penggunaan

```
embedding_model = SentenceTransformer('all-mpnet-base-v2')
df["content_embed"] = list(process_series_embedding(df["content"], model=embedding_model))
```



Kemudian, dataframe tersebut kami simpan dalam sebuah file agar proses embedding text tidak perlu dilakukan berulang-ulang. Pada environment kaggle yang dapat memanfaatkan penggunaan GPU untuk melakukan proses tersebut.

```
df["content_embed"].head()

0    [-0.014459913, -0.0506374, -0.0016422388, 0.01...
1    [0.033078566, -0.02033196, -0.02192107, 0.0153...
2    [0.05030278, -0.004991428, -0.0010037319, 0.04...
3    [0.033929758, -0.033017773, -0.034692187, -0.0...
4    [0.013225987, -0.053387415, -0.03501523, 0.016...
Name: content_embed, dtype: object

Save dataframe hasil embedding

# Simpan DataFrame ke file pickle
df.to_pickle("df_cyberattack.pkl")
```


kemudian dataframe tersebut kami bagi menjadi 3 bagian berupa train, test, validation dengan rasio 75:15:15. seperti snippet berikut

```
X = np.vstack(df_loaded['content_embed'])
y = df_loaded['class']

# Tentukan proporsi untuk train, validation, test secara manual
train_size = int(0.7 * len(X)) # 70% untuk training
val_size = int(0.15 * len(X)) # 15% untuk validation
test_size = len(X) - train_size - val_size # Sisanya untuk test

# Pisahkan 15% untuk test set
X_temp, X_test, y_temp, y_test = train_test_split(X, y, test_size=test_size, random_state=SEED)

# Pisahkan sisa 85% menjadi train dan validation set
X_train, X_val, y_train, y_val = train_test_split(X_temp, y_temp, test_size=val_size, random_state=SEED)
```

Lalu, dilakukan proses encoding untuk kolom label dari dataset tersebut menggunakan encoder one hot. yang menghasilkan encoding dalam NumPy array dan direshape.

```
# Fit OneHotEncoder untuk training data
encoder = OneHotEncoder(sparse_output=False, handle_unknown='ignore')

# Mengonversi y_train ke NumPy array dan reshape
y_train_onehot = encoder.fit_transform(y_train.values.reshape(-1, 1))

# Transform validation dan test data dengan fitted encoder
y_val_onehot = encoder.transform(y_val.values.reshape(-1, 1))
y_test_onehot = encoder.transform(y_test.values.reshape(-1, 1))
```

5. Model Implementation¶

Kemudian, kami mendeklarasikan model-model yang akan kami gunakan dalam melatih dataset dan menerapkan proses pipeline untuk skalabilitas proses training model tersebut.

```
# Definisikan model
models = {
    'Logistic Regression': LogisticRegression(max_iter=1000, random_state=SEED),
    'K-Nearest Neighbors': KNeighborsClassifier(n_neighbors=5),
    'Naive Bayes': GaussianNB(),
    'Decision Tree': DecisionTreeClassifier(random_state=SEED)
}

# Membuat pipeline (meskipun tidak diperlukan praproses di sini, pipeline memastikan skalabilitas)
pipelines = {name: Pipeline([(name, model)]) for name, model in models.items()}
```

6. Training Model

Setelah dilakukan proses training menggunakan pipeline seperti berikut, didapatkan hasil prediksi dari ke-4 model yang kami gunakan. Adapula Metric evaluasi yang digunakan sebagai acuan utama dalam menilai hasil training model berupa 'F1-Macro' dan 'Log Loss'.

```
# Simpan hasil evaluasi
results = {}

# Train & evaluasi pada validasi
for name, pipeline in pipelines.items():
    pipeline.fit(X_train, y_train) # Train dengan label asli
    y_val_pred = pipeline.predict(X_val)
    y_val_proba = pipeline.predict_proba(X_val)

    # Map prediksi kembali ke label asli
    y_val_true_labels = encoder.inverse_transform(y_val_onehot).ravel()
    y_val_pred_labels = y_val_pred # Sudah dalam format label asli

    conf_matrix = confusion_matrix(y_val_true_labels, y_val_pred_labels, labels=encoder.categories_[0])
    loss_val = log_loss(y_val_onehot, y_val_proba) # Gunakan one-hot encoded y untuk log_loss
    report = classification_report(y_val_true_labels, y_val_pred_labels, output_dict=True)

    results[name] = {
        'conf_matrix': conf_matrix,
        'log_loss': loss_val,
        'f1_macro': report['macro avg']['f1-score'],
        'classification_report': report
    }

# Display hasil validasi
print(f"\nValidation Results for {name}:")
print(f"Log Loss: {loss_val:.4f}")
print(f"F1-Macro: {report['macro avg']['f1-score']:.4f}")
print("\nClassification Report:")
print(classification_report(y_val_true_labels, y_val_pred_labels))
print("="*60)
```

Untuk model Logistic Regression didapatkan nilai F1-Macro: 0.028 dan Log Loss: 0.9963. Hasil tersebut menjadikan model Logistic Regression sebagai model yang relatif cukup baik pada posisi ke-2

terbaik dalam memprediksi kemungkinan pesan media sosial merupakan peringatan dini serangan *cyber*.

```
Validation Results for Logistic Regression:
Log Loss: 0.0280
F1-Macro: 0.9963

Classification Report:
              precision    recall  f1-score   support

 cyberattack         1.00        1.00        1.00        1064
      general         1.00        0.99        1.00         655

 accuracy                   1.00        1719
 macro avg          1.00        1.00        1.00        1719
 weighted avg       1.00        1.00        1.00        1719

=====
```

Kemudian, didapatkan hasil validasi untuk model KNN yaitu F1-Macro: 0.0096 dan Log Loss: 0.9969. Berdasarkan hasil tersebut didapat Model KNN dengan nilai Log Loss terendah dan F1-Macro tertinggi dari model yang dilatih lainnya. Sehingga dapat dikatakan bahwa model KNN ini merupakan model yang terbaik dalam memprediksi potensi pesan peringatan dini terjadinya serangan *cyber*

```
Validation Results for K-Nearest Neighbors:
Log Loss: 0.0096
F1-Macro: 0.9969

Classification Report:
              precision    recall  f1-score   support

 cyberattack         1.00        1.00        1.00        1064
      general         1.00        0.99        1.00         655

 accuracy                   1.00        1719
 macro avg          1.00        1.00        1.00        1719
 weighted avg       1.00        1.00        1.00        1719

=====
```

Untuk model Naive Bayes, didapatkan hasil F1-Macro sebesar 0.1894, dan Log Loss sebesar 0.9877. Berdasarkan hasil tersebut model Naive Bayes merupakan model ke-3 terbaik dibandingkan dengan model-model yang dilatih.

Validation Results for Naive Bayes:

Log Loss: 0.1894

F1-Macro: 0.9877

Classification Report:

	precision	recall	f1-score	support
cyberattack	0.99	0.99	0.99	1064
general	0.98	0.99	0.98	655
accuracy			0.99	1719
macro avg	0.99	0.99	0.99	1719
weighted avg	0.99	0.99	0.99	1719

=====

Terakhir, untuk model Decision Tree didapatkan hasil F1-Macro: 1.8452, dan nilai Log Loss: 0.9457. Ini merupakan model dengan hasil yang relatif kurang baik dibandingkan dengan model lainnya. Hal ini disebabkan karena tingginya nilai Log Loss dan rendahnya nilai F1-Macro pada model ini.

Validation Results for Decision Tree:

Log Loss: 1.8452

F1-Macro: 0.9457

Classification Report:

	precision	recall	f1-score	support
cyberattack	0.96	0.96	0.96	1064
general	0.93	0.93	0.93	655
accuracy			0.95	1719
macro avg	0.95	0.95	0.95	1719
weighted avg	0.95	0.95	0.95	1719

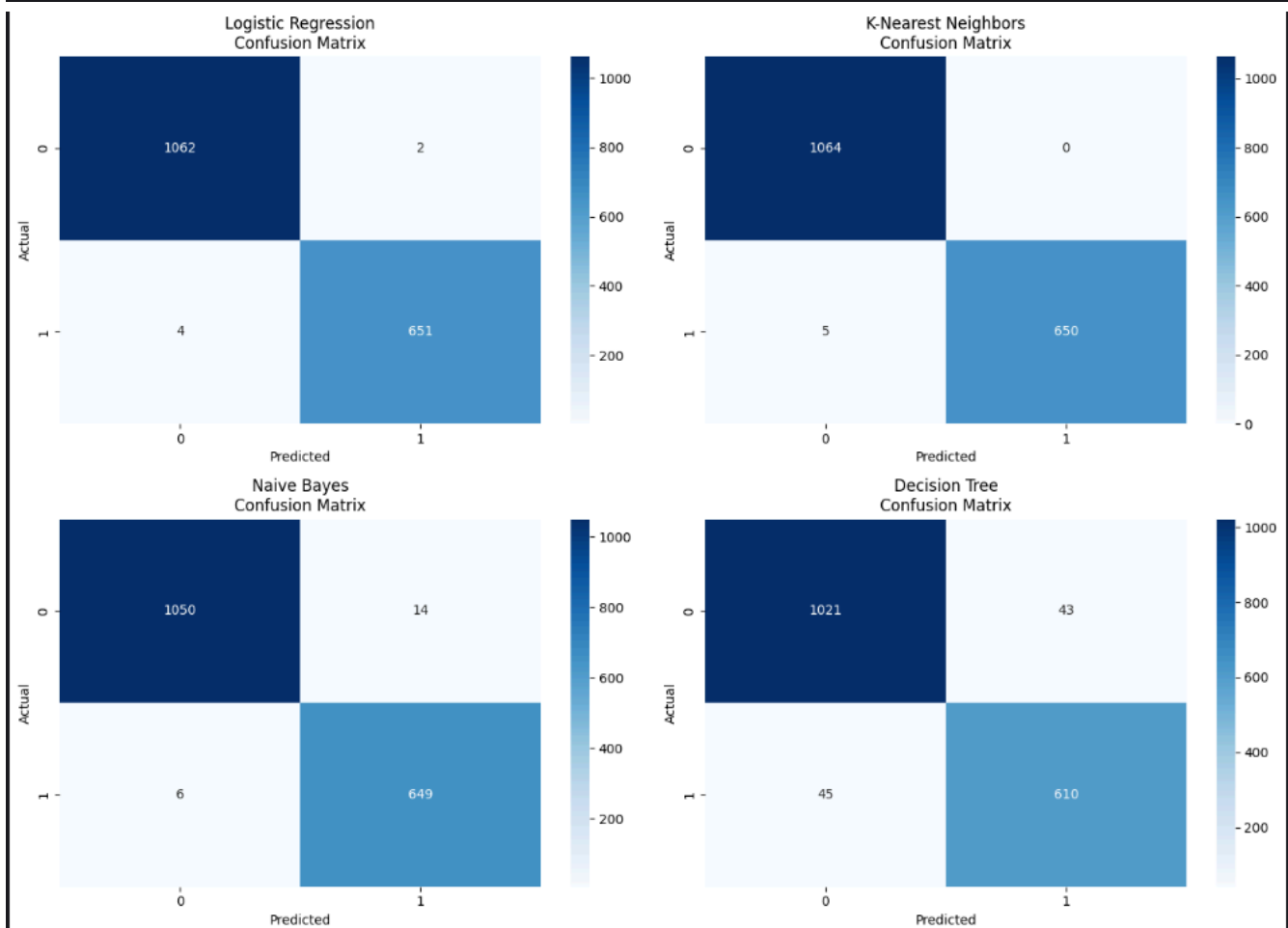
=====

7. Model Evaluation

Untuk melihat ilustrasi hasil training model tersebut secara lebih jelas, kami sajikan kembali dalam bentuk grafis agar perbandingan dapat dilakukan dengan lebih jelas. Dimana pada ke-empat model

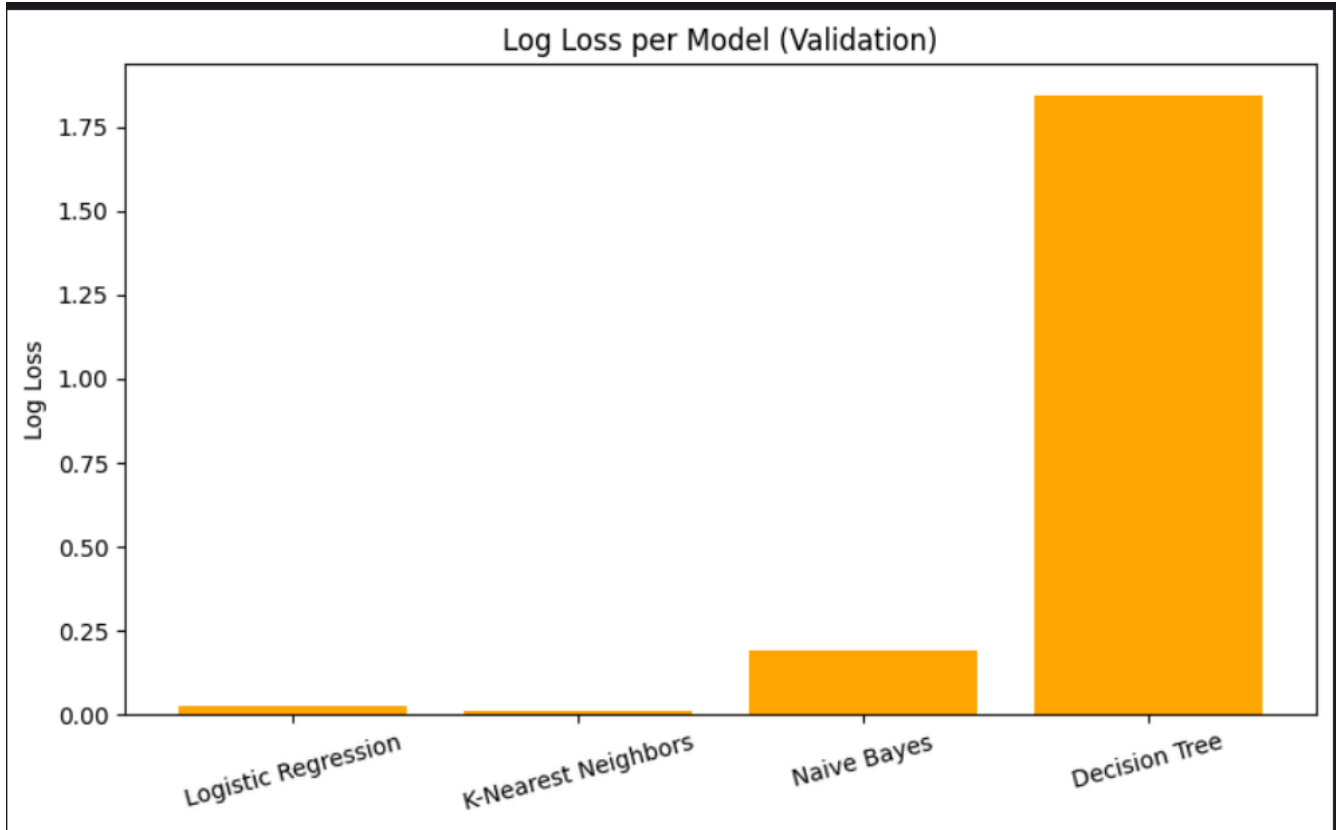
terlihat memiliki jumlah TP(True Positive), dan TN(True Negative) yang sangat bagus untuk model KNN, dan Logistic Regression. Namun terlihat bahwa model Naive Bayes memiliki jumlah data FP(False Positive), dan FN (False Negative) yang relatif lebih signifikan dibandingkan ke-2 model sebelumnya. Sedangkan untuk model Decision Tree tampak bahwa nilai FP, dan FN yang dimiliki memiliki perbedaan yang relatif sangat signifikan dibandingkan model yang dilatih lainnya.

```
# Plot confusion matrix
fig, axes = plt.subplots(2, 2, figsize=(14, 10))
axes = axes.flatten()
for ax, (name, res) in zip(axes, results.items()):
    sns.heatmap(res['conf_matrix'], annot=True, fmt='d', cmap='Blues', ax=ax)
    ax.set_title(f'{name}\nConfusion Matrix')
    ax.set_xlabel('Predicted')
    ax.set_ylabel('Actual')
plt.tight_layout()
```



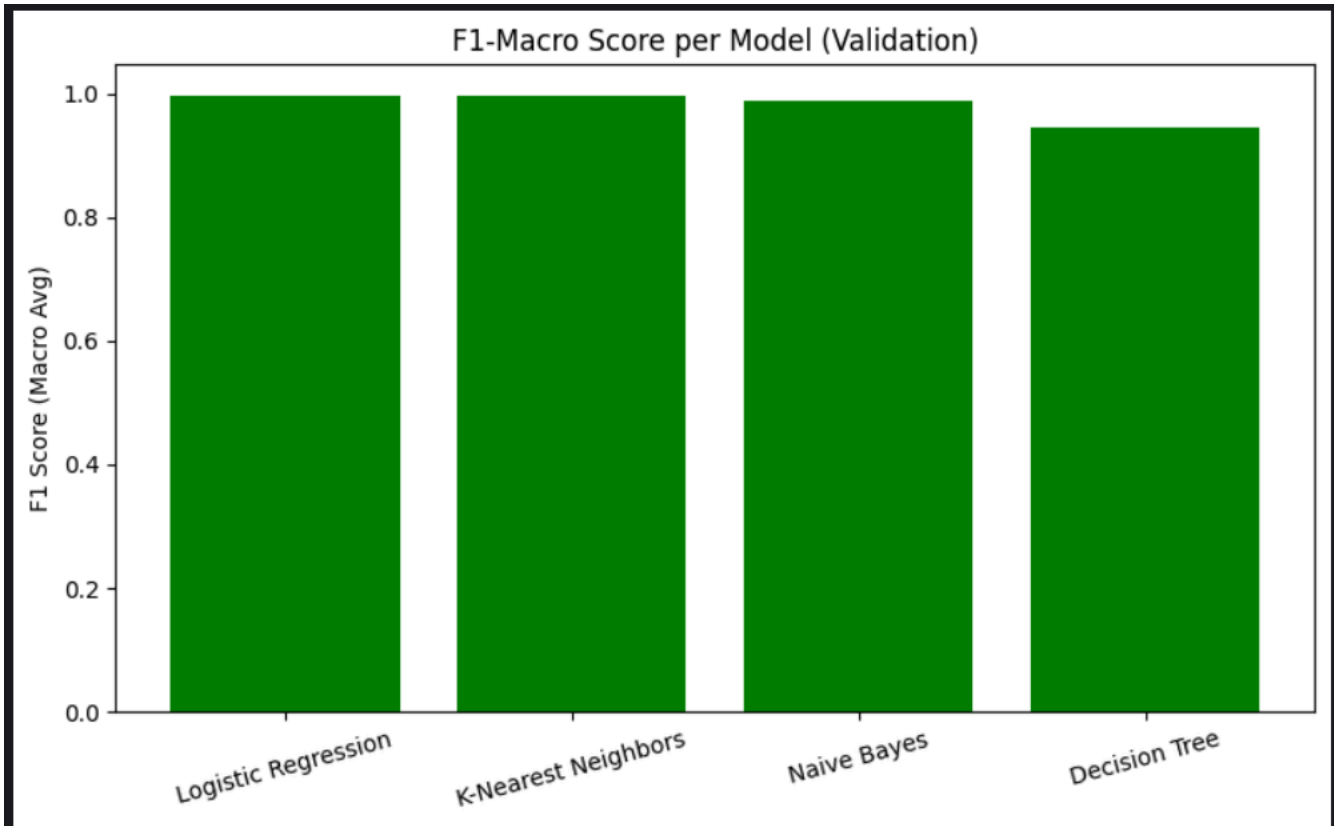
Untuk metric evaluasi Log Loss, dapat dilihat juga bahwa hasil yang didapat cukup mirip dengan hasil perbandingan jumlah FP, dan FN sebelumnya. Dimana didapat nilai Log Loss untuk model Logistic Regression berada sedikit lebih tinggi dibandingkan model KNN, relatif cukup tinggi untuk model Naive Bayes, dan untuk model Decision Tree didapat nilai yang relatif sangat tinggi.

```
# Plot log loss
plt.figure(figsize=(8, 5))
loss_values = [res['log_loss'] for res in results.values()]
plt.bar(results.keys(), loss_values, color='orange')
plt.ylabel('Log Loss')
plt.title('Log Loss per Model (Validation)')
plt.xticks(rotation=15)
plt.tight_layout()
```



Namun, pada ilustrasi nilai F1-Macro, tidak terlalu tampak perbedaan yang sangat jelas secara grafis. Hal tersebut mungkin disebabkan karena perbedaan nilai yang dihasilkan relatif kecil jika dibandingkan dengan skala pada grafis. Namun jika dilihat lebih teliti, hasil dari visualisasi nilai ini masih mengikuti tren perbedaan pada metric sebelumnya.

```
# Plot F1-Macro
plt.figure(figsize=(8, 5))
f1_scores = [res['f1_macro'] for res in results.values()]
plt.bar(results.keys(), f1_scores, color='green')
plt.ylabel('F1 Score (Macro Avg)')
plt.title('F1-Macro Score per Model (Validation)')
plt.xticks(rotation=15)
plt.tight_layout()
```



Berdasarkan hasil evaluasi tersebut, dapat disimpulkan bahwa model KNN merupakan model terbaik pada kasus ini. Oleh karena itu, kami melanjutkan untuk melakukan proses prediksi menggunakan model KNN untuk data test yang telah disediakan sebelumnya. Adapun untuk evaluasi dari data test ini memiliki nilai yang cenderung lebih rendah dibandingkan dengan data validation sebelumnya.

```

# Inference on test set (best model based on validation F1-macro)
best_model_name = max(results, key=lambda x: results[x]['f1_macro'])
best_pipeline = pipelines[best_model_name]
y_test_pred = best_pipeline.predict(X_test)
y_test_proba = best_pipeline.predict_proba(X_test)

# Map test set predictions back to original labels
y_test_true_labels = encoder.inverse_transform(y_test_onehot).ravel()
y_test_pred_labels = y_test_pred # Already in original label format

# Evaluate on test set
test_conf_matrix = confusion_matrix(y_test_true_labels, y_test_pred_labels, labels=encoder.categories_[0])
test_loss = log_loss(y_test_onehot, y_test_proba) # Use one-hot encoded y for log_loss
test_report = classification_report(y_test_true_labels, y_test_pred_labels, output_dict=True)

# Display test set results
print(f"\nInference Results for Best Model ({best_model_name}) on Test Set:")
print(f"Log Loss: {test_loss:.4f}")
print(f"F1-Macro: {test_report['macro avg']['f1-score']:.4f}")
print("\nClassification Report:")
print(classification_report(y_test_true_labels, y_test_pred_labels))

```

```

Inference Results for Best Model (K-Nearest Neighbors) on Test Set:
Log Loss: 0.0272
F1-Macro: 0.9963

```

Classification Report:

	precision	recall	f1-score	support
cyberattack	0.99	1.00	1.00	1053
general	1.00	0.99	1.00	668
accuracy			1.00	1721
macro avg	1.00	1.00	1.00	1721
weighted avg	1.00	1.00	1.00	1721

Begitu juga untuk nilai FP, dan FN yang dihasilkan pada data test ini. Dimana untuk jumlah persentase data yang sama, terdapat sedikit peningkatan untuk salah satu nilai tersebut.


```
# Plot test set confusion matrix
plt.figure(figsize=(6, 5))
sns.heatmap(test_conf_matrix, annot=True, fmt='d', cmap='Blues',
            xticklabels=encoder.categories_[0], yticklabels=encoder.categories_[0])
plt.title(f'{best_model_name}\nConfusion Matrix (Test)')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.tight_layout()
```

